



UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE ASTRONOMÍA

**STUDY OF THE ROLE OF GAS IN THE FORMATION OF SMBH SEED,
THROUGH SIMULATIONS OF NSC WITH GAS EMBEDDED**

TESIS PARA OPTAR AL GRADO DE MAGÍSTER EN CIENCIAS, MENCIÓN
ASTRONOMÍA

BORIS CUEVAS GOMEZ

PROFESOR GUÍA:
Andrés Escala

MIEMBROS DE LA COMISIÓN:
PROFESOR 2
PROFESOR 3

Este trabajo ha sido parcialmente financiado por:
NOMBRE INSTITUCIÓN

SANTIAGO DE CHILE
2026

RESUMEN DE LA MEMORIA PARA OPTAR
AL TÍTULO DE MAGÍSTER EN CIENCIAS,
MENCIÓN ASTRONOMÍA
POR: BORIS CUEVAS GOMEZ
FECHA: 2026
PROF. GUÍA: ANDRÉS ESCALA

**STUDY OF THE ROLE OF GAS IN THE FORMATION OF SMBH SEED,
THROUGH SIMULATIONS OF NSC WITH GAS EMBEDDED**

Abstractd

*Una frase de dedicatoria,
pueden ser dos líneas.*

Saludos

Acknowledgments

Acknowledgments

Table of Contents

1. Introduction	1
1.1. Formation Channels for SMBHs and the Collisional Paradigm	1
1.2. The Role of Gas in Dense Stellar Systems	1
1.3. Research Objectives and Thesis Outline	1
2. A scenario for SMBH formation	2
2.1. Density criterion for stellar collisions	2
2.2. Stars and gas in equilibrium	2
3. Numerical Methods	3
3.1. The AMUSE Framework	3
3.2. N-body code: Ph4	5
3.2.1. Hermite integrator	6
3.2.2. Block Time Steps	7
3.2.3. Collisions and mergers	8
3.2.4. Escaping stars	9
3.3. SPH code: Fi	10
3.3.1. Barnes-Hut tree	10
3.3.2. Smoothed Particle Hydrodynamics	11
3.4. Coupling Strategy: The Bridge Method	14
3.4.1. Symplectic Integrator	14
3.4.2. Bridge	16
4. Initial Conditions and Setup	18
4.1. Stellar Cluster Models	18
4.2. Gas Configurations	18
4.3. Numerical Parameters	18
5. Results and analysis	19
5.1. Gas-Free Baseline Simulations	19
5.2. Effects of Gas Fraction on Collision Rates	19
5.3. Mass Spectrum Evolution and Massive Object Formation	19
5.4. Comparative Analysis Across Gas Fractions	19
6. Discussion and Conclusions	20
6.1. Interpretation of Gas-Induced Enhancements	20
6.2. Implications for SMBH Seed Formation	20
6.3. Limitations and Future Work	20

References	21
Appendices	23
A. Fourth-Order Hermite Integration Scheme	23
A.1. Step 1: Particle selection	23
A.2. Step 2: Calculate the predicted position and velocities for all particles	23
A.3. Step 3: Calculate acceleration and jerk for particle i	24
A.4. Step 4: Second and third time derivative of acceleration, using Hermite interpolation	24
A.5. Step 5: Add corrections	25
A.6. Step 6: Time step update	26

List of Figures

3.1.	Schematic of the coupled numerical framework. The stellar N-body system (modelled with ph4) and the gaseous SPH system (modelled with Fi) interact gravitationally through the Bridge coupling scheme.	3
3.2.	Principal domains in AMUSE framework. Multiple physical problems can be solved coupling codes from different domains	4
3.3.	Schematic representation of the three-layer architecture of the AMUSE framework. The user interacts with the system via a Python script (top layer), which communicates through interfaces (middle layer) with the community codes (bottom layer).	5
3.4.	Representation of an oct tree	11
3.5.	Comparison of force calculation normalized (masses and gravitational constant set to 1). In dashed green, the exact gravitational force. In blue, the force with plummer softening, and in red, the force calculated with cubic spline softening.	17

Chapter 1

Introduction

- 1.1. Formation Channels for SMBHs and the Collisional Paradigm
- 1.2. The Role of Gas in Dense Stellar Systems
- 1.3. Research Objectives and Thesis Outline

Chapter 2

A scenario for SMBH formation

- 2.1. Density criterion for stellar collisions
- 2.2. Stars and gas in equilibrium

Chapter 3

Numerical Methods

This thesis investigates the role of gas in the formation of supermassive black hole (SMBH) seeds via stellar collisions in dense clusters. In contrast to previous studies, we do not include gas accretion onto stars, since we are interested exclusively in the gravitational interaction between gas and stars.

We require, in principle, three codes: 1) an N-body code to model the stellar dynamics, 2) a smoothed particle hydrodynamics (SPH) code to simulate the gas thermodynamics and self-gravity, and 3) a coupling scheme to manage the mutual gravitational interaction between these two components.

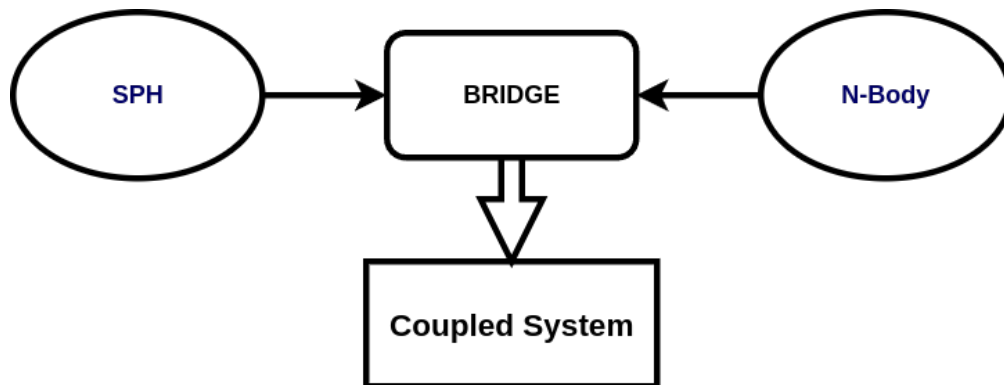


Figure 3.1: Schematic of the coupled numerical framework. The stellar N-body system (modelled with `ph4`) and the gaseous SPH system (modelled with `Fi`) interact gravitationally through the `Bridge` coupling scheme.

The N-Body and SPH codes are accessed through the AMUSE interface (Zwart and McMillan, 2018), and coupled via an external code.

3.1. The AMUSE Framework

The numerical codes used in this work are accessed through the Astrophysical Multipurpose Software Environment (AMUSE) (Zwart and McMillan, 2018). The AMUSE project aims to provide a flexible and modular framework for solving multi-physics problems in astrophysics. It offers a variety of community codes, classified into four main domains: gravitational dynamics, hydrodynamics, stellar evolution, and radiative transfer (Fig. 3.2). Some codes, such as GADGET-2 (Springel, 2005), can cover more than one domain.

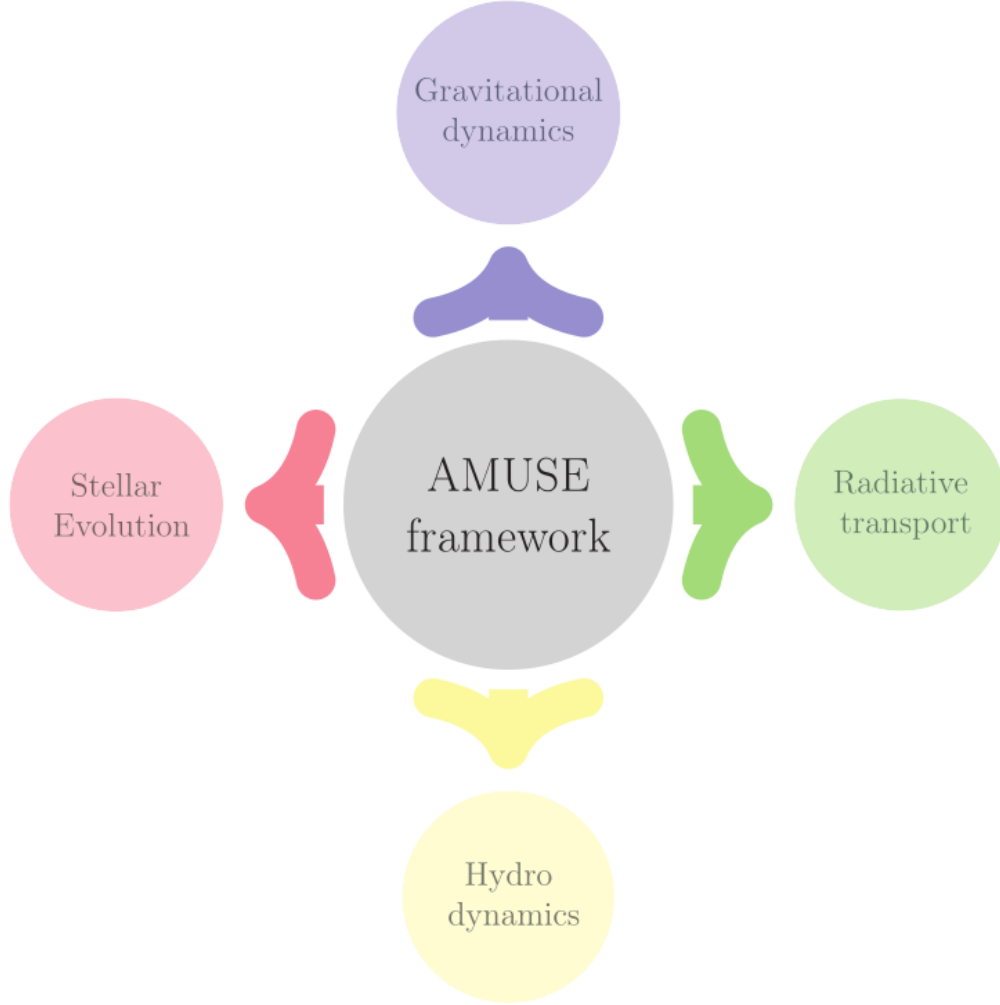


Figure 3.2: Principal domains in AMUSE framework. Multiple physical problems can be solved coupling codes from different domains

The structure of amuse is composed of three principal layers (Fig. 3.3). The top layer is the user interface, where a Python script is written to set up and run a specific scientific problem. In this script, codes are initialized, and additional routines (e.g., for handling stellar collisions) can be implemented. This layer also contains the manager, whose primary functions are to provide data structures and methods for communication between the user script and the community codes, as well as to handle unit conversions.

The middle layer provides the interfaces to the community codes. It facilitates bidirectional communication between the manager and the underlying code layer. Each interface consists of two components: a proxy, written in Python, which initializes the necessary functions and parameters, and a partner, written in the code's native language (e.g., C++, CUDA, or Fortran). The proxy and partner communicate via the Message Passing Interface (MPI).

The bottom layer contains the community codes themselves, implemented in their native languages (e.g., C++, CUDA, Fortran).

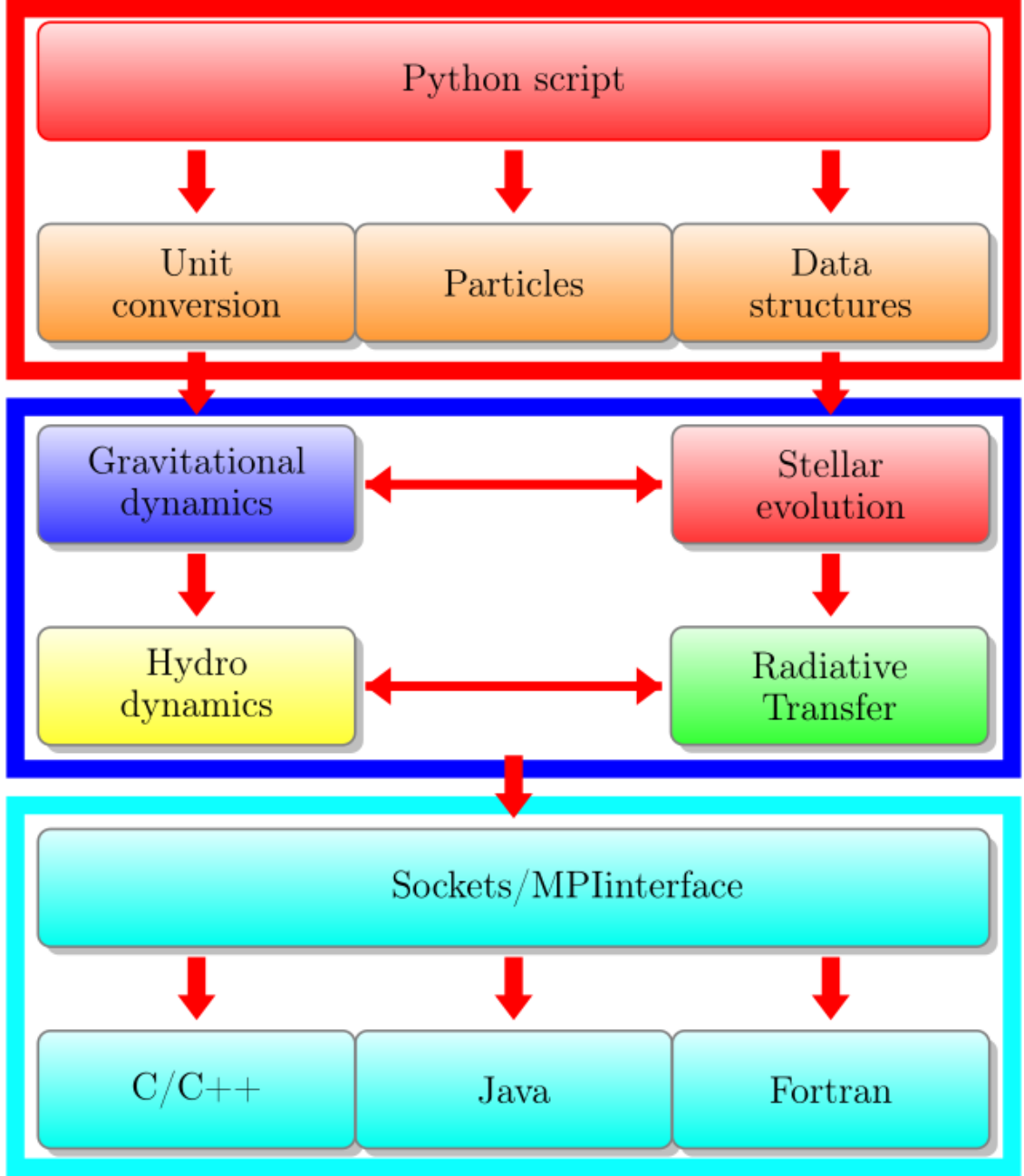


Figure 3.3: Schematic representation of the three-layer architecture of the AMUSE framework. The user interacts with the system via a Python script (top layer), which communicates through interfaces (middle layer) with the community codes (bottom layer).

3.2. N-body code: Ph4

The proposed scenarios presented in (referencia a capitulo) require high numerical precision to accurately model close encounters and direct collisions. We therefore employ the `ph4` N-body code from the AMUSE framework (Zwart and McMillan, 2018).

In this section, we include a description of the code, and some external routines.

3.2.1. Hermite integrator

The Ph4 code uses a fourth-order Hermite integrator (Makino and Aarseth, 1992) for calculating the positions and velocities of stars due to gravity. This is a predictor-corrector method.

The fourth-order Hermite scheme is based on a precise calculation of the individual time step. Considering the particle i with its own time t_i , time step Δt_i , position $\mathbf{x}_i$, velocity $\mathbf{v}_i$, acceleration $\mathbf{a}_i$ and jerk $\dot{\mathbf{a}}_i$, all calculated at time t_i . The integration algorithm proceeds as follows:

1. Select the particle with the minimum $t_i + \Delta t_i$. Set the global time to $t = t_i + \Delta t_i$.
2. Calculate the predicted positions ($\mathbf{x}_p$) and velocities ($\mathbf{v}_p$) for all particles, using the current values for $\mathbf{x}$, $\mathbf{v}$, $\mathbf{a}$, $\dot{\mathbf{a}}$.
3. Calculate the acceleration ($\mathbf{a}_i$) and jerk ($\dot{\mathbf{a}}_i$) for particle i at time $t_i + \Delta t_i$ using the predicted positions and velocities.
4. Calculate the second and third time derivatives of acceleration ($\mathbf{a}_i^{(2)}$ and $\mathbf{a}_i^{(3)}$) using Hermite interpolation.
5. Apply corrections to the position and velocity of particle i .
6. Calculate and update the time step Δt_i .
7. Repeat the algorithm.

A detailed derivation of the algorithm and the full set of equations are provided in Appendix A. From Eqs. (A.2) and (A.3), the predicted positions and velocities are:

$$\mathbf{x}_{p,j} = \mathbf{x}_j + \mathbf{v}_j(t - t_j) + \frac{\mathbf{a}_{0,j}}{2}(t - t_j)^2 + \frac{\dot{\mathbf{a}}_{0,j}}{3!}(t - t_j)^3 \quad (3.1)$$

$$\mathbf{v}_{p,j} = \mathbf{v}_j + \mathbf{a}_{0,j}(t - t_j) + \frac{\dot{\mathbf{a}}_{0,j}}{2!}(t - t_j)^2 \quad (3.2)$$

Here, $\mathbf{x}_j$ and $\mathbf{v}_j$ are the position and velocity of particle j at time t_j . The current acceleration $\mathbf{a}_{0,j}$ and jerk $\dot{\mathbf{a}}_{0,j}$ are computed as:

$$\mathbf{a}_{0,j} = \sum_{k \neq j} Gm_k \frac{\mathbf{r}_{0,jk}}{(\mathbf{r}_{0,jk}^2 + \epsilon^2)^{3/2}} \quad (3.3)$$

$$\dot{\mathbf{a}}_{0,j} = \sum_{k \neq j} Gm_k \left[\frac{\mathbf{v}_{0,jk}}{(\mathbf{r}_{0,jk}^2 + \epsilon^2)^{3/2}} - \frac{3(\mathbf{v}_{0,jk} \cdot \mathbf{r}_{0,jk})\mathbf{r}_{0,jk}}{(\mathbf{r}_{0,jk}^2 + \epsilon^2)^{5/2}} \right] \quad (3.4)$$

$$\mathbf{r}_{0,jk} = \mathbf{r}_k - \mathbf{r}_j \quad (3.5)$$

$$\mathbf{v}_{0,jk} = \mathbf{v}_k - \mathbf{v}_j \quad (3.6)$$

Using Hermite interpolation, the second and third time derivatives of the acceleration can be estimated. This estimation is subsequently used to correct the positions and velocities. The relevant expressions, derived in the appendix, are:

$$\mathbf{a}_{0,i}^{(2)} = \frac{-6(\mathbf{a}_{0,i} - \mathbf{a}_{1,i}) - \Delta t_i(4\dot{\mathbf{a}}_{0,i} + 2\dot{\mathbf{a}}_{1,i})}{\Delta t_i^2} \quad (3.7)$$

$$\mathbf{a}_{0,i}^{(3)} = \frac{12(\mathbf{a}_{0,i} - \mathbf{a}_{1,i}) + 6(\dot{\mathbf{a}}_{0,i} + \dot{\mathbf{a}}_{1,i})\Delta t_i}{\Delta t_i^3} \quad (3.8)$$

The final corrected position and velocity of particle i are then given by:

$$\mathbf{x}_i(t_i + \Delta t_i) = \mathbf{x}_{p,i} + \frac{\mathbf{a}_{0,i}^{(2)} \Delta t_i^4}{24} + \frac{\mathbf{a}_{0,i}^{(3)} \Delta t_i^5}{120} \quad (3.9)$$

$$\mathbf{v}_i(t_i + \Delta t_i) = \mathbf{v}_{p,i} + \frac{\mathbf{a}_{0,i}^{(2)} \Delta t_i^3}{6} + \frac{\mathbf{a}_{0,i}^{(3)} \Delta t_i^4}{24} \quad (3.10)$$

These corrections incorporate terms up to fourth order. The main advantage of this method is that it only requires the explicit calculation of the acceleration and jerk at each step, with higher-order derivatives being obtained through the interpolation scheme.

3.2.2. Block Time Steps

An important part of the Hermite integrator is the time step calculation. The time step controls precision, energy conservation and integration time. While time steps are individual (each particle has its own), they are grouped into blocks for efficiency.

Time steps are set in two steps. First, a dynamic time step is calculated. Then, the time step is assigned using a block scheme. For the dynamical time step, the standard formula (A.35) is used:

$$\Delta t_i = \sqrt{\eta \frac{|\mathbf{a}_{1,i}| |\mathbf{a}_{1,i}^{(2)}| + |\dot{\mathbf{a}}_{1,i}|^2}{|\dot{\mathbf{a}}_{1,i}| |\mathbf{a}_{1,i}^{(3)}| + |\mathbf{a}_{1,i}^{(2)}|^2}} \quad (3.11)$$

As can be seen, $\mathbf{a}_{1,i}$ and $\dot{\mathbf{a}}_{1,i}$ are calculated in each step, while $\mathbf{a}_{1,i}^{(3)}$ is constant. Only $\mathbf{a}_{1,i}^{(2)}$ needs to be estimated:

$$\mathbf{a}_{1,i}^{(2)} = \mathbf{a}_{0,i}^{(2)} + \Delta t_i \mathbf{a}_{1,i}^{(3)} \quad (3.12)$$

When the simulation begins, the initial time steps are calculated using (A.37):

$$\Delta t = \eta_s \frac{|\mathbf{a}|}{|\dot{\mathbf{a}}|} \quad (3.13)$$

The main algorithm of the Hermite 4th-order scheme evolves the particle with the minimum $t_j + \Delta t_j$. As it is expensive to evolve particles one by one, the goal is to assign particles to discrete time steps. In Ph4, time steps are set to powers of two of the N-body time units.

During initialization, the time step is calculated using Equation (3.13), and then set. Given the initial time step $\Delta t_{0,j}$, the N-body time τ , and an integer n :

$$0.5 < \frac{\tau 2^n}{\Delta t_{0,j}} < 1 \quad (3.14)$$

The time step is then set to $\tau 2^n$, with n being able to be positive or negative.

As the model evolves, the new time step is computed using (3.11). The time step is halved if the calculated time step is smaller than the current time step, and doubled if it is larger than twice the current time step.

The advantage of this time step criterion is that the time step in the code is equal to or lower than that required by the dynamics. Moreover, since particles share the same time steps, they can be integrated simultaneously. Additionally, all particles can be synchronized, as their times are multiples of powers of two of the same time unit.

3.2.3. Collisions and mergers

Stellar collisions occur in two steps. First, the collision is detected by the AMUSE routines in PH4, and then it is handled externally. PH4 has support for stopping conditions, specifically for collision detection.

All particles in AMUSE have different properties: position, velocity, mass, radius, among others. When `collision_detection` is enabled in PH4, the first filter is to check if two particles are closer than the sum of their radii:

$$|\Delta \mathbf{x}| < r_i + r_j \quad (3.15)$$

where $\Delta \mathbf{x}$ is the relative position vector between particles i and j , and r_i, r_j their respective radii. A second filter then verifies close encounters:

$$\epsilon |\Delta \mathbf{x}| |\Delta \mathbf{v}| > \Delta \mathbf{x} \cdot \Delta \mathbf{v} \quad (3.16)$$

where $\Delta \mathbf{v}$ is the relative velocity vector and $\epsilon = 0.001$. This avoids cases where particle superposition results from hyperbolic encounters with predominantly tangential relative velocities.

The stopping conditions allow the user to implement custom handling routines. For these simulations, we simply conserve linear momentum. When `collision_detection` is triggered, the code modifies the properties of one particle (preferentially the more massive one), setting its mass to the sum of the masses, its position to the center of mass, its velocity to the center-of-mass velocity, and its radius to conserve mass density:

$$m_{\text{new}} = m_i + m_j \quad (3.17)$$

$$\mathbf{x}_{\text{new}} = \frac{m_i \mathbf{x}_i + m_j \mathbf{x}_j}{m_i + m_j} \quad (3.18)$$

$$\mathbf{v}_{\text{new}} = \frac{m_i \mathbf{v}_i + m_j \mathbf{v}_j}{m_i + m_j} \quad (3.19)$$

$$r_{\text{new}} = r_i \left(\frac{m_i + m_j}{m_i} \right)^{1/3} \quad (3.20)$$

$$(3.21)$$

Finally, the other particle is removed from the simulation.

3.2.4. Escaping stars

After each analysis time step, potential escapers are identified through a multi-stage detection algorithm. To maintain mass and energy conservation, escaped stars are not removed from the simulation. Instead we add an extra attribute, **escaped** (all stars begin with **particles.escaped = False**), to track bound and unbound stars for subsequent analysis. The process begins with defining an appropriate length scale.

During initialization, the escape scale is defined as the minimum between ten times the virial radius and the Lagrangian radius containing the total mass:

$$r_{0,\text{esc}} = \min(10 \times r_{\text{vir}}, r_{100}) \quad (3.22)$$

where r_{100} represents the Lagrangian radius containing the total stellar mass. In subsequent iterations, the escape distance is updated as:

$$r_{\text{esc}} = \max(r_{\text{esc}}, 10 \times r_{b,\text{vir}}) \quad (3.23)$$

where $r_{b,\text{vir}}$ denotes the virial radius of bound stars.

The escape detection algorithm, executed at each time step, proceeds as follows:

1. Compute the core center and identify stars beyond the escape radius
2. Select gravitationally unbound stars
3. Identify stars with outward radial motion
4. Verify minimal gravitational influence on nearest neighbors

The core center location is determined using a density-weighted scheme following Sweatman (1993). For each star, the distance to its 7th nearest neighbor ($r_{i,7}$) is computed, and the density center is calculated as:

$$\mathbf{x}_c = \frac{\sum_{i=1}^N \mathbf{x}_i / r_{i,7}^3}{\sum_{i=1}^N 1 / r_{i,7}^3} \quad (3.24)$$

This formulation resembles a center of mass calculation but uses local stellar density instead of mass. Stars satisfying the condition

$$|\mathbf{x}_i - \mathbf{x}_c| > r_{\text{esc}} \quad (3.25)$$

are identified as potential escapers.

Gravitational unbinding is evaluated using the total energy $E_i = E_{p,i} + E_{k,i}$, where $E_{p,i}$ is the gravitational potential energy (including contributions from both stars and gas) and $E_{k,i}$ is the kinetic energy. Stars with $E_i \leq 0$ are considered bound, while those with $E_i > 0$ are classified as unbound. Among unbound stars, outward-moving candidates are identified through the condition:

$$\Delta \mathbf{x}_i \cdot \Delta \mathbf{v}_i > 0 \quad (3.26)$$

where $\Delta \mathbf{x}_i = \mathbf{x}_i - \mathbf{x}_c$ and $\Delta \mathbf{v}_i = \mathbf{v}_i - \mathbf{v}_c$ represent position and velocity relative to the cluster center. Similar to (3.24), the velocity of the core center is computed as

$$\mathbf{v}_c = \frac{\sum_{i=1}^N \mathbf{v}_i m_i / r_{i,7}^3}{\sum_{i=1}^N m_i / r_{i,7}^3} \quad (3.27)$$

The final validation step quantifies the gravitational influence on neighboring stars. For each candidate escaper, the relative potential energy ($e_{p,i}$) and kinetic energy ($e_{k,i}$) with respect to its nearest neighbor are computed. The energy ratio is defined as:

$$X_E := \frac{|e_{p,i}|}{e_{k,i}} \quad (3.28)$$

Stars satisfying $X_E < 0.01$ are confirmed as escapers, ensuring minimal gravitational perturbation to neighboring stars. These stars are tagged as escapers (`particles.escaped = True`).

3.3. SPH code: Fi

The gas dynamics are modelled using the **Fi** code (Gerritsen et al., 1997), an MPI-parallel smoothed particle hydrodynamics (SPH) implementation within the AMUSE framework. We focus on the aspects most relevant to our simulations; a comprehensive description of the code can be found in (Pelupessy, 2005).

3.3.1. Barnes-Hut tree

A key feature of **Fi** is its ability to compute the self-gravity of the gaseous component. Particle-based codes offer a range of methods for gravitational force calculation. The most accurate approach is the direct summation of all pairwise interactions, which scales as $\mathcal{O}(N^2)$ and is computationally expensive for large N . While such precision is often necessary for collisional stellar dynamics (e.g., in our N-body component), the gas system in our simulations is treated as a collisionless fluid for the purpose of gravity calculation, allowing for faster, approximate methods, sacrificing accuracy for computational efficiency.

Fi employs the Barnes-Hut (BH) tree algorithm (Barnes and Hut, 1986). This method uses a hierarchical oct-tree spatial decomposition and a geometric opening criterion to approximate gravitational forces.

The tree is constructed by enclosing all particles within a root cell. If this cell contains more than one particle, it is subdivided into eight daughter cells of equal volume. This subdivision process repeats recursively for each cell containing multiple particles, until every cell holds at most one particle (a leaf node). Figure 3.4 illustrates this structure in two dimensions.

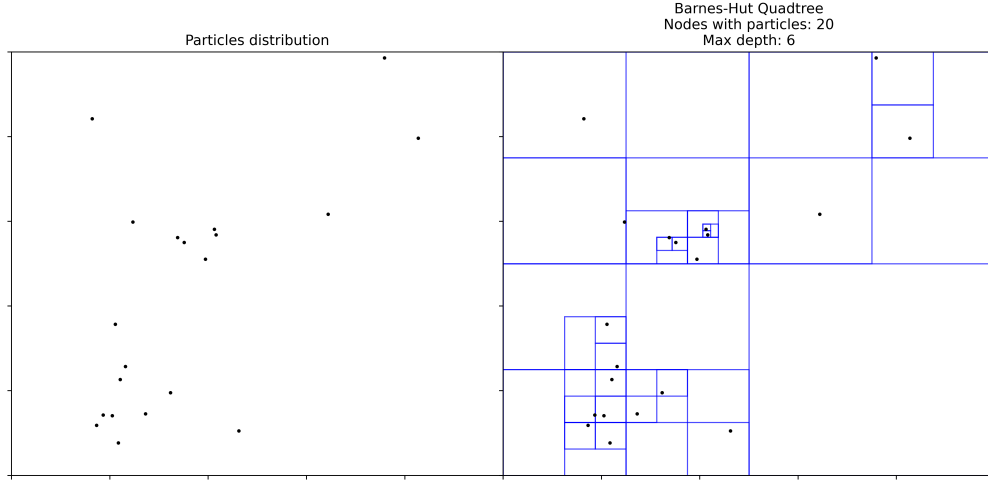


Figure 3.4: Representation of an oct tree

Once the tree is built, the gravitational force on a given particle is calculated by going through the tree from the root. For each node encountered (linked to a cell), a decision is made: either open the node to examine its children, or accept the node as a single interacting entity. The classic Barnes-Hut opening criterion is:

$$l/d < \theta \quad (3.29)$$

where l is the side length of the cubic cell, d is the distance from the target particle to the node's center of mass, and θ is an accuracy parameter. If this condition is met, the entire mass of the node is treated as a single point mass at its center of mass, providing an acceptable approximation when the node is sufficiently distant (i.e., appears small in angular size). The recursion stops upon reaching a leaf node, which by definition represents a single particle and is interacted with directly. This approximation reduces the computational cost of the force calculation from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log N)$ and adapts efficiently to clustered mass distributions, making it suitable for collisionless systems (Barnes and Hut, 1986).

The standard geometric criterion (Eq. 3.29), however, can fail in some circumstances (Salmon and Warren, 1994). Therefore, **Fi** implements a modified criterion following (Springel and Hernquist, 2002):

$$M_n l^4 < \alpha |\mathbf{a}| d^6 \quad (3.30)$$

where M_n is the total mass contained in the node, $\mathbf{a}$ is an estimate of the gravitational acceleration (typically from the previous timestep), and α is a parameter to control accuracy.

3.3.2. Smoothed Particle Hydrodynamics

The gas dynamics are solved using the Smoothed Particle Hydrodynamics (SPH) method (Gingold and Monaghan, 1977; Lucy, 1977). In SPH, the hydrodynamic equations are solved from a Lagrangian perspective, representing the fluid with a discrete set of particles rather than a continuous medium. For an inviscid fluid described by density ρ , pressure p , and specific internal energy u , the Lagrangian takes the form:

$$L = \int \left[\frac{1}{2} v^2 - u(\rho) \right] \rho d^3 \mathbf{x}, \quad (3.31)$$

where the specific internal energy satisfies $\partial u / \partial \rho = p / \rho^2$, and $\mathbf{r} = (x_1, x_2, x_3)$ is the position vector. From this Lagrangian, the Euler equations of motion can be derived via a variational principle (Rasio, 2000):

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) = 0 \quad (3.32)$$

$$\frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla) \mathbf{v} = -\frac{\nabla p}{\rho} \quad (3.33)$$

$$\frac{\partial u}{\partial t} + (\mathbf{v} \cdot \nabla) u = -\frac{p}{\rho} (\nabla \cdot \mathbf{v}) \quad (3.34)$$

Equations (3.32), (3.33), and (3.34) are the continuity, momentum, and energy equations, respectively.

To obtain a discrete representation, the continuous Lagrangian is replaced by a particle-based analogue:

$$L_{\text{SPH}} = \sum_{i=1}^N m_i \left[\frac{1}{2} \mathbf{v}_i^2 - u_i \right], \quad (3.35)$$

where m_i , $\mathbf{v}_i$, and ρ_i are the mass, velocity, and density of each particle (representing a fluid element), and $u_i = u(\rho_i)$ is the specific internal energy. To use Eq. (3.35) as the system's Lagrangian, a suitable expression for the density in terms of the particle coordinates is required.

A key ingredient of the method is the density estimate. At any point $\mathbf{r}$, the density is computed by interpolating over neighboring particles:

$$\rho(\mathbf{r}) = \sum_j m_j W(\mathbf{r} - \mathbf{r}_j; h), \quad (3.36)$$

where $W(\mathbf{r} - \mathbf{r}_j; h)$ is a smoothing kernel of characteristic width h (the smoothing length). The kernel must satisfy two conditions: its integral over all space must be unity, and it should approach a Dirac delta function in the limit $h \rightarrow 0$. The first condition ensures that integrating Eq. (3.36) over space recovers the total mass of the system.

In the case of **Fi**, the cubic spline kernel is adopted (Monaghan, 1992; Pelupessy, 2005):

$$W(r, h) = \frac{1}{\pi h^3} \begin{cases} 1 - \frac{3}{2} \left(\frac{r}{h}\right)^2 + \frac{3}{4} \left(\frac{r}{h}\right)^3, & 0 \leq \frac{r}{h} \leq 1, \\ \frac{1}{4} \left(2 - \frac{r}{h}\right)^3, & 1 \leq \frac{r}{h} \leq 2, \\ 0, & \text{otherwise.} \end{cases} \quad (3.37)$$

With the density estimation and kernel defined, we now turn to the treatment of the smoothing length h . In modern SPH implementations, it is common to use an adaptive smoothing length h_i for each particle, adjusted to keep the number of neighboring particles approximately constant. However, the use of adaptive smoothing lengths introduces ∇h terms in the equations, which can lead to violations of energy and entropy conservation if not properly accounted for (Nelson and Papaloizou, 1994). A rigorous way to incorporate variable smoothing lengths is to treat h_i as additional variables in the Lagrangian formulation, subject

to constraints (Springel and Hernquist, 2002). In **Fi**, the following constraint is imposed:

$$\frac{4\pi}{3}h_i^3(\rho_i + \tilde{\rho}) = M_{\text{SPH}}, \quad (3.38)$$

where $M_{\text{SPH}} = \bar{m}N_{\text{nb}}$, with $\bar{m}$ the mean particle mass and N_{nb} the target number of neighbors. The term $\tilde{\rho}$ is an extra density introduced to prevent h_i from diverging as $\rho_i \rightarrow 0$ (Pelupessy, 2005). This constraint can be incorporated into the variational principle via Lagrangian multipliers, yielding correction factors in the equations of motion (see AppendixB for the full derivation).

With the notation $W_{ij}(h_i) \equiv W(|\mathbf{r}_i - \mathbf{r}_j|, h_i)$ and the symmetrized kernel $\bar{W}_{ij} = [W_{ij}(h_i) + W_{ij}(h_j)]/2$, the equation of motion for particle i becomes:

$$\frac{d\mathbf{v}_i}{dt} = - \sum_{j=1}^N m_j \left[f_i \frac{P_i}{\rho_i^2} \nabla_i W_{ij}(h_i) + f_j \frac{P_j}{\rho_j^2} \nabla_i W_{ij}(h_j) + \Pi_{ij} \nabla_i \bar{W}_{ij} \right], \quad (3.39)$$

where the correction factor f_i accounts for the variability of the smoothing length:

$$f_i = \left(1 + \frac{h_i}{3(\rho_i + \tilde{\rho})} \frac{\partial \rho_i}{\partial h_i} \right)^{-1}. \quad (3.40)$$

To handle discontinuities such as shocks, an artificial viscosity (AV) term Π_{ij} is included. In SPH, entropy generation is confined to shocks through this viscous term. **Fi** implements the standard Monaghan artificial viscosity (Monaghan, 1992; Pelupessy, 2005):

$$\Pi_{ij} = \frac{-\alpha \mu_{ij} \bar{c}_{ij} + \beta \mu_{ij}^2}{\bar{\rho}_{ij}} \quad (3.41)$$

with

$$\mu_{ij} = \begin{cases} \frac{\mathbf{v}_{ij} \cdot \mathbf{r}_{ij}}{h_{ij}(\mathbf{r}_{ij}^2/h_{ij}^2 + \eta^2)}, & \text{for } \mathbf{v}_{ij} \cdot \mathbf{r}_{ij} < 0, \\ 0, & \text{for } \mathbf{v}_{ij} \cdot \mathbf{r}_{ij} \geq 0 \end{cases} \quad (3.42)$$

where $\mathbf{v}_{ij} = \mathbf{v}_i - \mathbf{v}_j$ and $\mathbf{r}_{ij} = \mathbf{r}_i - \mathbf{r}_j$. The dimensionless parameters α and β control the strength of the viscosity (default values are $\alpha = 0.5$, $\beta = 1.0$), and $\eta \ll 1$ prevents numerical divergences. Quantities overlaid with a bar are symmetrized averages (e.g., $\bar{c}_{ij} = (c_i + c_j)/2$ is the average sound speed). The AV is only activated when particles are approaching each other ($\mathbf{v}_{ij} \cdot \mathbf{r}_{ij} < 0$), ensuring that $\mu_{ij} \leq 0$ and therefore $\Pi_{ij} \geq 0$. As the term $-\nabla_i W_{ij}(h_i)$ is proportional to $(\mathbf{r}_i - \mathbf{r}_j) = \mathbf{r}_{ij}$ (see AppendixB), the AV acts to decrease kinetic energy, with the dissipated energy converted into thermal energy, thereby increasing the internal energy (or entropy) of the particles.

Finally, the thermodynamic evolution of the gas must be specified. **Fi** offers two formulations: evolving either the specific internal energy u_i or the entropic function A_i (related to the specific entropy). For an ideal gas with adiabatic index γ , these quantities are related through

$$P_i = A_i \rho_i^\gamma, \quad (3.43)$$

$$u_i = \frac{A_i}{\gamma - 1} \rho_i^{\gamma-1} = \frac{P_i}{(\gamma - 1) \rho_i}, \quad (3.44)$$

where A_i is the entropic function, which remains constant for adiabatic processes in the absence of shocks.

The evolution equation for the entropic function is

$$\frac{dA_i}{dt} = -\frac{\gamma-1}{\rho_i^\gamma}(\Gamma - \Lambda) + \frac{1}{2} \frac{\gamma-1}{\rho_i^{\gamma-1}} \sum_{j=1}^N m_j \Pi_{ij} \mathbf{v}_{ij} \cdot \nabla_i \bar{W}_{ij} \quad (3.45)$$

where Γ and Λ represent external heating and cooling rates (both set to zero in our simulations). Alternatively, the internal energy equation can be integrated:

$$\frac{du_i}{dt} = \sum_{j=1}^N m_j \left[\frac{f_i p_i}{2 \rho_i^2} \nabla_i W_{ij}(h_i) + \frac{f_j p_j}{2 \rho_j^2} \nabla_i W_{ij}(h_j) + \Pi_{ij} \nabla_i \bar{W}_{ij} \right] \mathbf{v}_{ij} \quad (3.46)$$

For our simulations, we adopt the energy equation (Eq. (3.46)), as it provides better numerical stability in the coupled stellar-gas framework (see Section (por hacer) for a detailed discussion).

3.4. Coupling Strategy: The Bridge Method

The N-body and SPH codes operate as independent modules and must be coupled for mixed star-gas simulations. A key advantage of using SPH for the gas component is that both systems are represented by particles, which simplifies the implementation of gravitational interactions. In this section, we describe the coupling scheme employed in this work, based on the Bridge method (Fujii et al., 2007) and follows the implementation of Reinoso Reinoso (2024).

3.4.1. Symplectic Integrator

The Bridge method (Fujii et al., 2007) employs a symplectic integrator, specifically the second-order leapfrog scheme. The evolution of a dynamical system can be described by Hamilton's equations:

$$\frac{df}{dt} = \{f, H\}, \quad (3.47)$$

where f, H denotes the Poisson bracket:

$$\{f, H\} = \sum_i \left(\frac{\partial f}{\partial q_i} \frac{\partial H}{\partial p_i} - \frac{\partial f}{\partial p_i} \frac{\partial H}{\partial q_i} \right) = D_H f(t) \quad (3.48)$$

with (q_i, p_i) being the canonical coordinates. The formal solution of this equation is given by the exponential operator:

$$f(t) = e^{tD_H} f(0) \quad (3.49)$$

For a gravitational Hamiltonian of the form

$$H(\mathbf{x}, \mathbf{p}) = \frac{\mathbf{p}^2}{2m} + \Phi(\mathbf{x}) \equiv H_{\text{kin}}(\mathbf{p}) + H_{\text{pot}}(\mathbf{x}), \quad (3.50)$$

the Liouville operator D_H can be decomposed as:

$$D_H = \frac{\partial H}{\partial \mathbf{p}} \cdot \frac{\partial}{\partial \mathbf{x}} - \frac{\partial H}{\partial \mathbf{x}} \cdot \frac{\partial}{\partial \mathbf{p}} \quad (3.51)$$

$$= \frac{\mathbf{p}}{m} \cdot \frac{\partial}{\partial \mathbf{x}} + (-\nabla \Phi(\mathbf{x})) \cdot \frac{\partial}{\partial \mathbf{p}} \quad (3.52)$$

$$\equiv D_{\text{drift}} + D_{\text{kick}}. \quad (3.53)$$

The solution after a time step Δt is then:

$$f(t + \delta t) = e^{\delta t D_H} f(t) \quad (3.54)$$

When applied to the phase-space coordinates $(\mathbf{x}, \mathbf{v})$, the exponential operator $e^{\delta t D_H}$ has no closed-form expression due to the coupling between position and velocity (Zwart and McMillan, 2018). However, the individual operators D_{drift} and D_{kick} have simple actions:

$$D_{\text{drift}}(\mathbf{x}, \mathbf{v}) = (\mathbf{v}, \mathbf{0}), \quad (3.55)$$

$$D_{\text{kick}}(\mathbf{x}, \mathbf{v}) = (\mathbf{0}, -\nabla \Phi(\mathbf{x})). \quad (3.56)$$

The drift operator advances positions at constant velocity, while the kick operator updates velocities at constant position. Importantly, these operators do not commute: $[D_{\text{drift}}, D_{\text{kick}}] \neq 0$. However, the exponential operator can be approximated using an operator splitting technique (Fujii et al., 2007):

$$e^{\Delta t D_H} = e^{\delta t (D_{\text{kick}} + D_{\text{drift}})} = \prod_{i=1}^k e^{a_i \delta t D_{\text{kick}}} e^{b_i \delta t D_{\text{drift}}} + \mathcal{O}(\Delta t^{k+1}). \quad (3.57)$$

For $k = 2$ with coefficients $a_1 = a_2 = 1/2$, $b_1 = 1$, $b_2 = 0$, we obtain the second-order accurate approximation:

$$e^{\delta t (D_{\text{kick}} + D_{\text{drift}})} \approx e^{(\delta t/2) D_{\text{kick}}} e^{\delta t D_{\text{drift}}} e^{(\delta t/2) D_{\text{kick}}} + \mathcal{O}(\delta t^3). \quad (3.58)$$

This leads to the kick-drift-kick leapfrog integrator, which proceeds as follows for a time step Δt :

1. Kick: Advance velocities by $\Delta t/2$ using accelerations computed at the current positions:

$$\mathbf{v}_0 \rightarrow \mathbf{v}_{1/2} = \mathbf{v}_0 + \frac{\Delta t}{2} \mathbf{a}_0$$

2. Drift: Advance positions by Δt using the new half-step velocities:

$$\mathbf{x}_0 \rightarrow \mathbf{x}_1 = \mathbf{x}_0 + \Delta t \mathbf{v}_{1/2}$$

3. Kick: Advance velocities by another $\Delta t/2$ using accelerations computed at the updated positions:

$$\mathbf{v}_{1/2} \rightarrow \mathbf{v}_1 = \mathbf{v}_{1/2} + \frac{\Delta t}{2} \mathbf{a}_1$$

where $\mathbf{a}_0 = \mathbf{a}(\mathbf{x}_0)$ and $\mathbf{a}_1 = \mathbf{a}(\mathbf{x}_1)$ are the accelerations due to the gravitational potential.

The subscripts 0, 1/2, and 1 denote quantities evaluated at times t , $t + \Delta t/2$, and $t + \Delta t$, respectively.

3.4.2. Bridge

AMUSE includes an implementation of the original Bridge scheme. An example coupling an N-body code with an SPH code can be found in the AMUSE distribution at:

`amuse/examples/simple/gas_in_cluster.py`

This example models a star cluster with an embedded gas component, coupling GADGET-2 (SPH) and HERMITE (N-body). In this implementation, each code performs its own drift step internally. The kick step, however, is handled externally as follows:

- Accelerations $\mathbf{a}_{\text{NB},i}$ for each N-body particle due to all SPH particles are computed using the SPH code's `get_gravity_at_point(eps, x, y, z)` method, which returns the gravitational acceleration at a given point (x, y, z) with softening length *eps*. The velocities of N-body particles are then kicked using these accelerations.
- Similarly, accelerations $\mathbf{a}_{\text{SPH},i}$ for each SPH particle due to all N-body particles are computed using the `get_gravity_at_point(eps, x, y, z)` method of a tree code (BHTree in the example), with the softening length set to the SPH smoothing length h_i . The velocities of SPH particles are then kicked accordingly.

As identified by Reinoso Reinoso (2024), this implementation has two main issues, both related to the kick steps. First, SPH and N-body particles use different gravitational softening lengths (h and ϵ , respectively), leading to asymmetric forces:

$$F_{\text{NB}} = \frac{Gm_{\text{NB}}m_{\text{SPH}}}{(r^2 + \epsilon^2)^{3/2}}r \neq F_{\text{SPH}} = \frac{Gm_{\text{NB}}m_{\text{SPH}}}{(r^2 + h^2)^{3/2}}r, \quad (3.59)$$

where $r = |\mathbf{r}_{\text{NB}} - \mathbf{r}_{\text{SPH}}|$. This asymmetry can be resolved by introducing an effective softening length $\bar{\epsilon}$ for the mutual gravitational interaction during the kick steps.

Second, the force calculation may use different softening kernels: SPH codes often employ a cubic spline softening, while N-body codes typically use a Plummer (or similar) softening. The difference between these softening formulations is illustrated in Figure 3.5.

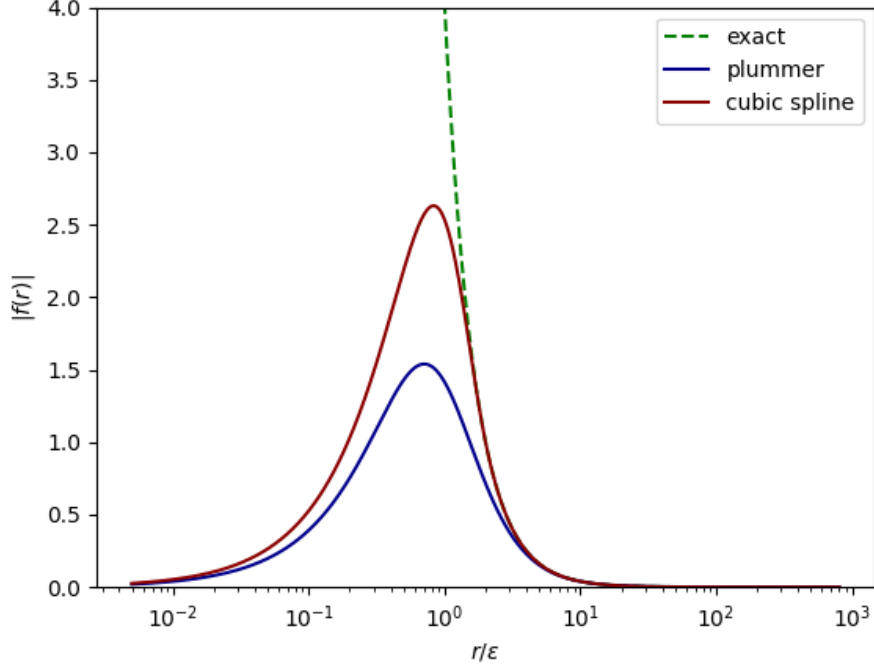


Figure 3.5: Comparison of force calculation normalized (masses and gravitational constant set to 1). In dashed green, the exact gravitational force. In blue, the force with plummer softening, and in red, the force calculated with cubic spline softening.

To handle these issues, Reinoso Reinoso (2024) developed a modified Bridge implementation that uses the **FASTKICK** code to compute gravitational interactions between SPH and N-body particles. **FASTKICK** computes forces using a Plummer softening with a constant softening length $\bar{\epsilon}$, ensuring symmetry and consistency. The code provides a Python interface, making it compatible with AMUSE simulation scripts, with the computationally intensive force calculations performed in compiled languages (C/Fortran), and a GPU-accelerated version (C++/CUDA) is also available.

The modified Bridge algorithm proceeds as follows for each global time step Δt :

1. At time t , compute accelerations $\mathbf{a}_{\text{NB},i}$ for all N-body particles due to SPH particles, and $\mathbf{a}_{\text{SPH},i}$ for all SPH particles due to N-body particles, using **FASTKICK** with softening length $\bar{\epsilon}$. Kick velocities by $\Delta t/2$.
2. Evolve each code independently for a full time step Δt (drift), updating positions via their own internal integrators.
3. Recompute accelerations using **FASTKICK** with the updated positions, and apply a second velocity kick of $\Delta t/2$.

Chapter 4

Initial Conditions and Setup

In chapter 3 a description of the numerical method is provided, including the SPH code for gas, the N-body code for stars and the coupling method adopted. In order to have a full specification of the simulations, this chapter provides a description of the parameters used in the codes initialization, and the initial conditions for gas and stars systems.

4.1. Plummer distribution

4.2. Gas Configurations

4.3. Numerical Parameters

Chapter 5

Results and analysis

- 5.1. Gas-Free Baseline Simulations
- 5.2. Effects of Gas Fraction on Collision Rates
- 5.3. Massive Object Formation
- 5.4. Comparative Analysis Across Gas Fractions

Chapter 6

Discussion and Conclusions

- 6.1. Interpretation of Gas-Induced Enhancements
- 6.2. Implications for SMBH Seed Formation
- 6.3. Limitations and Future Work

References

- Aarseth, S. J. (1985). Direct methods for n-body simulations. In Brackbill, J. U. and Cohen, B. I., editors, *Multiple Time Scales*, pages 377–418. Academic Press.
- Barnes, J. and Hut, P. (1986). A hierarchical $O(N \log N)$ force-calculation algorithm. *nature*, 324(6096):446–449.
- Fujii, M., Iwasawa, M., Funato, Y., and Makino, J. (2007). Bridge: A direct-tree hybrid-body algorithm for fully self-consistent simulations of star clusters and their parent galaxies. *Publications of the Astronomical Society of Japan*, 59(6):1095–1106.
- Gerritsen, J., Icke, V., et al. (1997). Star formation in n-body simulation, i. *Astronomy & Astrophysics*, 325.
- Gingold, R. A. and Monaghan, J. J. (1977). Smoothed particle hydrodynamics: theory and application to non-spherical stars. *Monthly notices of the royal astronomical society*, 181(3):375–389.
- Lucy, L. B. (1977). A numerical approach to the testing of the fission hypothesis. *Astronomical Journal*, vol. 82, Dec. 1977, p. 1013-1024., 82:1013–1024.
- Makino, J. and Aarseth, S. J. (1992). On a hermite integrator with ahmad-cohen scheme for gravitational many-body problems. *Publications of the Astronomical Society of Japan*, 44(2):141–151.
- Monaghan, J. J. (1992). Smoothed particle hydrodynamics. In: *Annual review of astronomy and astrophysics*. Vol. 30 (A93-25826 09-90), p. 543-574., 30:543–574.
- Nelson, R. P. and Papaloizou, J. C. (1994). Variable smoothing lengths and energy conservation in smoothed particle hydrodynamics. *Monthly Notices of the Royal Astronomical Society*, 270(1):1–20.
- Pelupessy, F. I. (2005). *Numerical studies of the interstellar medium on galactic scales*. PhD thesis, éditeur non identifié.
- Rasio, F. A. (2000). Particle methods in astrophysical fluid dynamics. *Progress of Theoretical Physics Supplement*, 138:609–621.
- Reinoso Reinoso, B. A. (2024). *Formation of massive black hole seeds through runaway stellar collisions and gas accretion in dense stellar systems*. PhD thesis.
- Salmon, J. K. and Warren, M. S. (1994). Skeletons from the treecode closet. *Journal of Computational Physics*, 111(1):136–155.
- Springel, V. (2005). The cosmological simulation code gadget-2. *Monthly notices of the royal astronomical society*, 364(4):1105–1134.
- Springel, V. and Hernquist, L. (2002). Cosmological smoothed particle hydrodynamics

- simulations: the entropy equation. *Monthly Notices of the Royal Astronomical Society*, 333(3):649–664.
- Sweatman, W. L. (1993). A study of Lagrangian radii oscillations and core-wandering using N-body simulations. *MNRAS*, 261(3):497–512.
- Zwart, S. P. and McMillan, S. (2018). *Astrophysical Recipes: the art of AMUSE*. IOP Publishing.

Appendices

Appendix A. Fourth-Order Hermite Integration Scheme

The N-body code used in this work, **ph4**, is based on a fourth-order Hermite integrator. The following subsections describe in detail the individual time step scheme (ITS) from Makino and Aarseth (1992), including the time step criterion in the N-body code.

A.1. Step 1: Particle selection

The first step is to select the next particle to be advanced. At any given time, each particle j has its own mass m_j , time t_j , time step Δt_j , coordinates $\mathbf{x}_j$ and $\mathbf{v}_j$, acceleration $\mathbf{a}_j$ and jerk $\dot{\mathbf{a}}_j$ (with $\dot{\mathbf{a}}_j = \partial \mathbf{a}_j / \partial t$). The selected particle i is the one satisfying the condition:

$$i = \arg \min_j (t_j + \Delta t_j) \quad (\text{A.1})$$

Note that always $t_j \leq t_i + \Delta t_i$, for all j . The current simulation time is then set to $t = t_i + \Delta t_i$.

A.2. Step 2: Calculate the predicted position and velocities for all particles

The positions and velocities of all particles are predicted at time t , using a Taylor expansion up to third-order (jerk):

$$\mathbf{x}_{p,j} = \mathbf{x}_j + \mathbf{v}_j(t - t_j) + \frac{\mathbf{a}_{0,j}}{2!}(t - t_j)^2 + \frac{\dot{\mathbf{a}}_{0,j}}{3!}(t - t_j)^3 \quad (\text{A.2})$$

$$\mathbf{v}_{p,j} = \mathbf{v}_j + \mathbf{a}_{0,j}(t - t_j) + \frac{\dot{\mathbf{a}}_{0,j}}{2!}(t - t_j)^2 \quad (\text{A.3})$$

The acceleration is due only to gravitational force. The expression for each particle j is:

$$\mathbf{a}_{0,j} = \sum_{k \neq j} G m_k \frac{\mathbf{r}_{0,jk}}{(\mathbf{r}_{0,jk}^2 + \epsilon^2)^{3/2}} \quad (\text{A.4})$$

where ϵ is the softening parameter, and

$$\mathbf{r}_{0,jk} = \mathbf{r}_k - \mathbf{r}_j \quad (\text{A.5})$$

Differentiating Eq. (A.4) with respect to time yields the jerk:

$$\dot{\mathbf{a}}_{0,j} = \frac{\partial \mathbf{a}_j}{\partial t} \quad (\text{A.6})$$

$$= \sum_{k \neq j} Gm_k \frac{\partial}{\partial t} \frac{\mathbf{r}_{0,jk}}{(\mathbf{r}_{0,jk}^2 + \epsilon^2)^{3/2}} \quad (\text{A.7})$$

$$\dot{\mathbf{a}}_{0,j} = \sum_{k \neq j} Gm_k \left[\frac{\mathbf{v}_{0,jk}}{(\mathbf{r}_{0,jk}^2 + \epsilon^2)^{3/2}} - \frac{3(\mathbf{v}_{0,jk} \cdot \mathbf{r}_{0,jk})\mathbf{r}_{0,jk}}{(\mathbf{r}_{0,jk}^2 + \epsilon^2)^{5/2}} \right] \quad (\text{A.8})$$

where

$$\mathbf{v}_{0,jk} = \mathbf{v}_k - \mathbf{v}_j \quad (\text{A.9})$$

A.3. Step 3: Calculate acceleration and jerk for particle i

Similar to equations (A.4) and (A.8), the acceleration and jerk are calculated using the predicted positions and velocity:

$$\mathbf{a}_{1,i} = \sum_{j \neq i} Gm_j \frac{\mathbf{r}_{ij}}{(\mathbf{r}_{ij}^2 + \epsilon^2)^{3/2}} \quad (\text{A.10})$$

$$\dot{\mathbf{a}}_{1,i} = \sum_{j \neq i} Gm_j \left[\frac{\mathbf{v}_{ij}}{(\mathbf{r}_{ij}^2 + \epsilon^2)^{3/2}} - \frac{3(\mathbf{v}_{ij} \cdot \mathbf{r}_{ij})\mathbf{r}_{ij}}{(\mathbf{r}_{ij}^2 + \epsilon^2)^{5/2}} \right] \quad (\text{A.11})$$

$$\mathbf{r}_{ij} = \mathbf{r}_{p,j} - \mathbf{r}_{p,i} \quad (\text{A.12})$$

$$\mathbf{v}_{ij} = \mathbf{v}_{p,j} - \mathbf{v}_{p,i} \quad (\text{A.13})$$

In this notation, the subscript 0 denotes quantities evaluated at the beginning of the time step (t_j), while the subscript 1 denotes quantities evaluated at the end of the time step (t), using the predicted position and velocity ($\mathbf{x}_{p,i}$ and $\mathbf{v}_{p,i}$).

A.4. Step 4: Second and third time derivative of acceleration, using Hermite interpolation

To obtain a high-order correction of $\mathbf{x}$ and $\mathbf{v}$, the acceleration of particle i is modeled as a third-order polynomial in time:

$$\mathbf{a}(\tau) = \alpha\tau^3 + \beta\tau^2 + \gamma\tau + \delta \quad (\text{A.14})$$

The successive time derivatives of $\mathbf{a}(\tau)$ are then:

$$\dot{\mathbf{a}}(\tau) = 3\alpha\tau^2 + 2\beta\tau + \gamma \quad (\text{A.15})$$

$$\mathbf{a}^{(2)}(\tau) = 6\alpha\tau + 2\beta \quad (\text{A.16})$$

$$\mathbf{a}^{(3)}(\tau) = 6\alpha \quad (\text{A.17})$$

The coefficients α , β , γ , and δ are determined by using the known values of acceleration and jerk at times t_i and t . This provides a system of four equations:

$$\mathbf{a}(t_i) = \mathbf{a}_{0,i} = \alpha t_i^3 + \beta t_i^2 + \gamma t_i + \delta \quad (\text{A.18})$$

$$\mathbf{a}(t) = \mathbf{a}_{1,i} = \alpha t^3 + \beta t^2 + \gamma t + \delta \quad (\text{A.19})$$

$$\dot{\mathbf{a}}(t_i) = \dot{\mathbf{a}}_{0,i} = 3\alpha t_i^2 + 2\beta t_i + \gamma \quad (\text{A.20})$$

$$\dot{\mathbf{a}}(t) = \dot{\mathbf{a}}_{1,i} = 3\alpha t^2 + 2\beta t + \gamma \quad (\text{A.21})$$

where $t = t_i + \Delta t_i$ is the current time. As the system of equations is linearly independent, the four coefficients can be determined. However, determining α and β is enough to get the second and third time derivative of acceleration. From (A.21):

$$\gamma = \dot{\mathbf{a}}_{1,i} - 3\alpha t^2 - 2\beta t \quad (\text{A.22})$$

Subtracting Eq. (A.20) from (A.21):

$$\beta = \frac{(\dot{\mathbf{a}}_{1,i} - \dot{\mathbf{a}}_{0,i}) - \alpha 3(t^2 - t_i^2)}{2(t - t_i)} \quad (\text{A.23})$$

Subtracting Eq. (A.18) from (A.19), and using results from (A.22) and (A.23) gives an expression for α :

$$\alpha = \frac{(\dot{\mathbf{a}}_{1,i} + \dot{\mathbf{a}}_{0,i})(t - t_i) - 2(\mathbf{a}_{1,i} - \mathbf{a}_{0,i})}{(t - t_i)^3} \quad (\text{A.24})$$

Replacing this result in Eq. (A.23):

$$\beta = \frac{-\dot{\mathbf{a}}_{1,i}(t + 2t_i) - \dot{\mathbf{a}}_{0,i}(2t + t_i)}{(t - t_i)^2} + \frac{3(\mathbf{a}_{1,i} - \mathbf{a}_{0,i})(t + t_i)}{(t - t_i)^3} \quad (\text{A.25})$$

The second and third time derivatives of the acceleration at time t_i can now be evaluated. For the Eq. (A.17), and using $t - t_i = \Delta t_i$:

$$\mathbf{a}_{0,i}^{(3)} = \mathbf{a}^{(3)}(t_i) = 6\alpha \quad (\text{A.26})$$

$$\mathbf{a}_{0,i}^{(3)} = 6 \frac{(\dot{\mathbf{a}}_{1,i} + \dot{\mathbf{a}}_{0,i})\Delta t_i - 2(\mathbf{a}_{1,i} - \mathbf{a}_{0,i})}{\Delta t_i^3} \quad (\text{A.27})$$

$$\mathbf{a}_{0,i}^{(3)} = \frac{12(\mathbf{a}_{0,i} - \mathbf{a}_{1,i}) + 6(\dot{\mathbf{a}}_{0,i} + \dot{\mathbf{a}}_{1,i})\Delta t_i}{\Delta t_i^3} \quad (\text{A.28})$$

Evaluating the second time derivative in t_i , and replacing the obtained for α and β :

$$\mathbf{a}_{0,i}^{(2)} = \mathbf{a}^{(2)}(t_i) = 6\alpha t_i + \beta \quad (\text{A.29})$$

$$\mathbf{a}_{0,i}^{(2)} = \frac{-6(\mathbf{a}_{0,i} - \mathbf{a}_{1,i}) - \Delta t_i(4\dot{\mathbf{a}}_{0,i} + 2\dot{\mathbf{a}}_{1,i})}{\Delta t_i^2} \quad (\text{A.30})$$

A.5. Step 5: Add corrections

The next step in the scheme is to correct position and velocity of particle i at time t . Using a higher-order Taylor expansion around t_i :

$$\mathbf{x}_i(t_i + \Delta t_i) = \mathbf{x}_i(t_i) + \dot{\mathbf{x}}_i(t_i)\Delta t_i + \frac{\mathbf{x}_i^{(2)}(t_i)\Delta t_i^2}{2!} + \dots + \frac{\mathbf{x}_i^{(5)}(t_i)\Delta t_i^5}{5!} \quad (\text{A.31})$$

$$= \mathbf{x}_i + \mathbf{v}_i\Delta t_i + \frac{\mathbf{a}_{0,i}\Delta t_i^2}{2} + \frac{\dot{\mathbf{a}}_{0,i}\Delta t_i^3}{6} + \frac{\mathbf{a}_{0,i}^{(2)}\Delta t_i^4}{24} + \frac{\mathbf{a}_{0,i}^{(3)}\Delta t_i^5}{120} \quad (\text{A.32})$$

The first four terms of this expansion correspond to the predicted position $\mathbf{x}_{p,i}$ from Eq. (A.2). Therefore, the corrected position can be written as:

$$\mathbf{x}_i(t_i + \Delta t_i) = \mathbf{x}_{p,i} + \frac{\mathbf{a}_{0,i}^{(2)}\Delta t_i^4}{24} + \frac{\mathbf{a}_{0,i}^{(3)}\Delta t_i^5}{120} \quad (\text{A.33})$$

In a similar way, the corrected velocity is:

$$\mathbf{v}_i(t_i + \Delta t_i) = \mathbf{v}_{p,i} + \frac{\mathbf{a}_{0,i}^{(2)}\Delta t_i^3}{6} + \frac{\mathbf{a}_{0,i}^{(3)}\Delta t_i^4}{24} \quad (\text{A.34})$$

The advantage of this scheme is that achieving high integration accuracy only requires the explicit calculation of the acceleration and jerk for all particles, and the predicted acceleration and jerk for the particle being advanced. This results in an integration scheme that is fourth-order accurate in time ($\mathcal{O}(a^{(3)})$).

A.6. Step 6: Time step update

After updating the position and velocity of particle i , a new time step Δt_i must be calculated. This step is crucial for controlling the integration error. A proven and stable criterion is given by the standard formula Aarseth (1985):

$$\Delta t_i = \sqrt{\eta \frac{|\mathbf{a}_{1,i}| |\mathbf{a}_{1,i}^{(2)}| + |\dot{\mathbf{a}}_{1,i}|^2}{|\dot{\mathbf{a}}_{1,i}| |\mathbf{a}_{1,i}^{(3)}| + |\mathbf{a}_{1,i}^{(2)}|^2}} \quad (\text{A.35})$$

The values for $\mathbf{a}_{1,i}$ and $\dot{\mathbf{a}}_{1,i}$ are known from the direct calculation in Step 3. The value of $\mathbf{a}^{(3)}(t)$ is constant, since a third-order polynomial interpolation is used. Only $\mathbf{a}_{1,i}^{(2)}$ should be calculated:

$$\mathbf{a}_{1,i}^{(2)} = \mathbf{a}_{0,i}^{(2)} + \Delta t_i \mathbf{a}_{0,i}^{(3)} \quad (\text{A.36})$$

During the initialization of the algorithm, the higher-order derivatives of the acceleration are not available, so an alternative formula for time step can be used in the startup:

$$\Delta t = \eta_s \frac{|\mathbf{a}|}{|\dot{\mathbf{a}}|} \quad (\text{A.37})$$

The suggested value for the startup parameter is $\eta_s \sim 0.01$ Makino and Aarseth (1992).